

SymPy Bug Report

1 Bug 39: Limit of Bessel quotient derivative is negative infinity

1.1 Minimal Reproducer

```
from sympy import besselj, diff, limit, series, symbols

x = symbols("x")
expr = diff(besselj(1, x) / x, x)

print("actual limit:", limit(expr, x, 0))
print("expected limit:", 0)
print("series of derivative:", series(expr, x, 0, 5))
```

1.2 Observed Behavior

```
actual limit: -oo
expected limit: 0
series of derivative: -x/8 + x**3/96 + O(x**5)
```

SymPy reports that the one-sided limit at the origin is negative infinity, even though SymPy's own local series for the differentiated expression starts with a linear term and therefore tends to 0. The returned value is not a harmless alternate representation of the same limit: $-\infty$ and 0 are mathematically distinct.

1.3 Expected Behavior

The mathematically correct result is

$$\lim_{x \rightarrow 0} \frac{d}{dx} \left(\frac{J_1(x)}{x} \right) = 0,$$

so the call `limit(diff(besselj(1, x) / x, x), x, 0)` should return 0.

1.4 Mathematical Explanation

The Bessel function has the Taylor expansion

$$J_1(x) = \frac{x}{2} - \frac{x^3}{16} + \frac{x^5}{384} + O(x^7).$$

Dividing by x gives an analytic even series at the origin,

$$\frac{J_1(x)}{x} = \frac{1}{2} - \frac{x^2}{16} + \frac{x^4}{384} + O(x^6).$$

Therefore

$$\frac{d}{dx} \left(\frac{J_1(x)}{x} \right) = -\frac{x}{8} + \frac{x^3}{96} + O(x^5),$$

which tends to 0 as $x \rightarrow 0$. The expression is analytic after the removable singularity in $J_1(x)/x$ is resolved, so an infinite limit is a genuine mathematical error rather than a branch-choice or simplification issue.

1.5 Source-Level Diagnosis

The diagnosis identifies three relevant SymPy source files:

```
sympy/functions/special/bessel.py
sympy/series/limits.py
sympy/core/add.py
```

The public call first differentiates $\text{besselj}(1, x)/x$ through `BesselBase.fdiff`; that derivative is correct. The resulting expression is then processed by the limit code, `Limit.doit`. It treats the expression as meromorphic at zero and asks for a leading term. That request reaches the additive leading-term method:

```
Add._eval_as_leading_term
```

The failure appears in this method’s cancellation fallback. For the differentiated Bessel quotient, the expanded expression contains the pieces

$$\frac{J_0(x)}{2x}, \quad -\frac{J_1(x)}{x^2}, \quad -\frac{J_2(x)}{2x}.$$

Their initial leading terms include $1/(2x)$ and $-1/(2x)$, which cancel. The fallback correctly detects that cancellation but then requests an `_eval_nseries` expansion of order $n = 0$. According to the diagnosis, that order is too low: the truncated Bessel expansion still exposes a spurious $-1/(2x)$ term, while a higher-order expansion reveals the true next term $-x/8$. `Limit.doit` receives the false negative-power leading term and converts it to `-oo`.

```
n0 = min.getn()
...
while res.is_Order:
    res = old._eval_nseries(x, n=n0+incr, logx=logx, cdir=cdir).cancel().powsimp().
    trigsimp()
    incr *= 2
return res.as_leading_term(x, logx=logx, cdir=cdir)
```

A plausible fix direction supported by the diagnosis is to make the additive leading-term fallback request enough additional series terms after a leading order cancels before accepting a non-`Order` result. In this case, an expansion to at least order 2 exposes the correct leading term $-x/8$.

1.6 Additional Failing Instances

The independent verification checks the representative case and five scaled instances of

$$\lim_{x \rightarrow 0} \frac{d}{dx} \left(\frac{J_1(cx)}{x} \right).$$

For each positive rational or integer scale c , the same Taylor expansion argument applies:

$$\frac{J_1(cx)}{x} = \frac{c}{2} - \frac{c^3 x^2}{16} + O(x^4), \quad \frac{d}{dx} \left(\frac{J_1(cx)}{x} \right) = -\frac{c^3 x}{8} + O(x^3).$$

Thus every listed limit should be 0. The verification also evaluates the derivative numerically near zero using `mpmath` and the identity $J_1'(z) = (J_0(z) - J_2(z))/2$, which agrees with the $-c^3 x/8$ leading behavior.

Input / Expression	SymPy behavior	Expected behavior
$\lim_{x \rightarrow 0} \frac{d}{dx} (J_1(x)/x)$	Returns <code>-oo</code>	Returns 0
$\lim_{x \rightarrow 0} \frac{d}{dx} (J_1(x/2)/x)$	Returns <code>-oo</code>	Returns 0
$\lim_{x \rightarrow 0} \frac{d}{dx} (J_1(3x/2)/x)$	Returns <code>-oo</code>	Returns 0
$\lim_{x \rightarrow 0} \frac{d}{dx} (J_1(3x)/x)$	Returns <code>-oo</code>	Returns 0
$\lim_{x \rightarrow 0} \frac{d}{dx} (J_1(4x)/x)$	Returns <code>-oo</code>	Returns 0
$\lim_{x \rightarrow 0} \frac{d}{dx} (J_1(5x)/x)$	Returns <code>-oo</code>	Returns 0

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new failure mode with no public precedent. The closest related public materials identified by the artifact were SymPy's limits documentation and source-documentation mirrors for the Gruntz and Bessel subsystems. Those sources provide background for the relevant machinery, but the artifact did not identify a public issue or discussion for this exact Bessel quotient, the exact `-oo` result, or a fix that would necessarily make this cancellation visible to `limit`. The bug therefore remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Rational, besselj, diff, limit, symbols

@pytest.mark.parametrize("c", [1, Rational(1, 2), Rational(3, 2), 3, 4, 5])
def test_limit_derivative_scaled_besselj1_quotient(c):
    x = symbols("x")
    expr = diff(besselj(1, c * x) / x, x)

    assert limit(expr, x, 0) == 0
```